

iter_R4_grow_05_10_15_25

Iterative contact-seeded inference for MarFold 1B

**Lifts mean LDDT on the FoldBench-10 train set from 0.250 → 0.351
(+40.7%)**

Issue #27 · exp27 · branch exp/27-improved-inference-algorithm

The problem

- MarFold 1B is a Llama-arch LLM trained on protein documents in the `contacts-and-distances-v1` grammar:

```
<begin_sequence> <AAs> <begin_statements>  
  [<*-range-contact> <p_i> <p_j>]*  
  <distance> <p_i> <p_j> <atom_i> <atom_j> -> <d_X.X>
```

- **Baseline readout** (exp20): prompt with `<begin_statements>`, then for every (i, j) pair read the next-token distribution over `<d_X.X>` bins. Zero context, no autoregression, no seeded contacts.
- On the 10-protein FoldBench train set: **mean LDDT = 0.2496** (median 0.250, per-protein 0.151–0.449).
- **Goal:** clear **+50%** over baseline (mean LDDT ≥ 0.3744), within $5\times$ the baseline wall-clock (≤ 6920 s on one A100).

Key insight: the model is great with context

Baseline readout

```
<begin_statements>  
<distance> <p_i> <p_j>  
  <CB> <CB>
```

mean LDDT
0.250

Seeded contacts

```
<begin_statements>  
<long-range-contact>  
  ...  
<medium-...> ...  
<distance> ...
```

mean LDDT
0.282 (+13%)

GT oracle (diagnostic)

```
<begin_statements>  
ALL true contacts  
  (gt_d < 8 Å)  
<distance> ...
```

mean LDDT
0.717 (+187%)

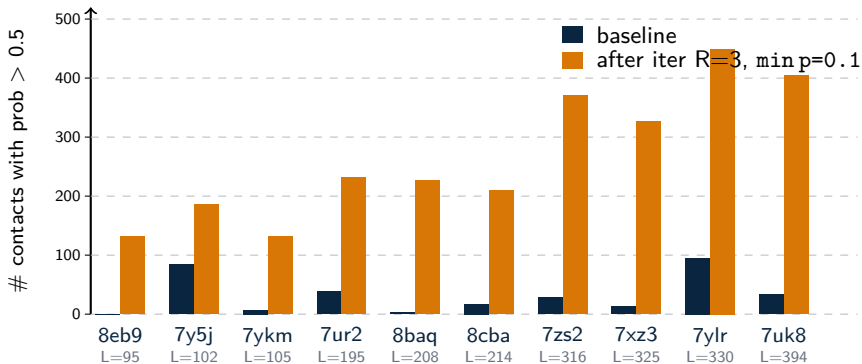
→ contact-prediction quality is the bottleneck for honest algorithms.

How do we get honest, high-quality contacts at inference time?

Idea: iterate. The model's distance readout under a seeded prefix is sharper

→ extract NEW high-confidence contacts from the round- N distogram → re-seed.

Iteration enriches the high-confidence contact pool



8baq: 3 $\rightarrow$ 227 contacts (76 $\times$). **8eb9:** 0 $\rightarrow$ 132. Every protein gets a much richer high-confidence pool after iteration.

$\rightarrow$ **later rounds can afford LARGER K without losing precision.**

The algorithm: iter_R4_grow_05_10_15_25

Round-by-round contact budget

$K = k_{\text{per } L} \cdot L$ per round:

R1  $K = 0.5 \cdot L$

R2  $K = 1.0 \cdot L$

R3  $K = 1.5 \cdot L$

R4  $K =$

Each round picks fresh seeds from the previous round's (sharper) distogram.

Pseudocode:

```
distogram = naive_readout()

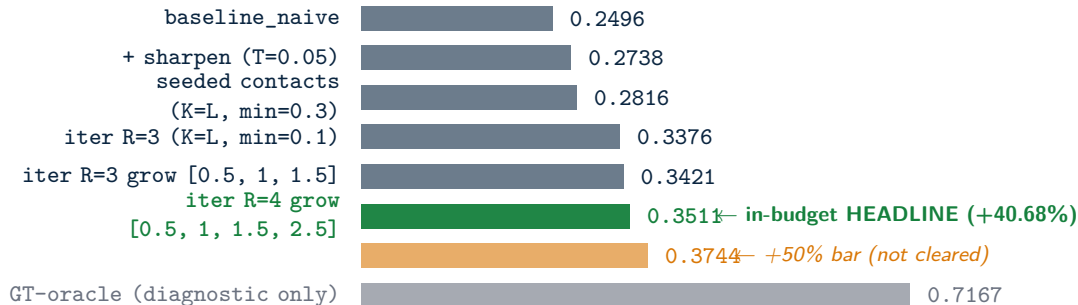
for K in [0.5L, 1.0L, 1.5L, 2.5L]:
    seeds = top-K contacts from
           distogram where  $p > 0.1$ ,
           ordered long > medium > short

    prefix = <begin_statements>
            + [<{range}-range-contact>
              <p_i> <p_j>
              for seed in seeds]

    distogram = readout under
                prefix, only for
                LDDT-shell pairs
```

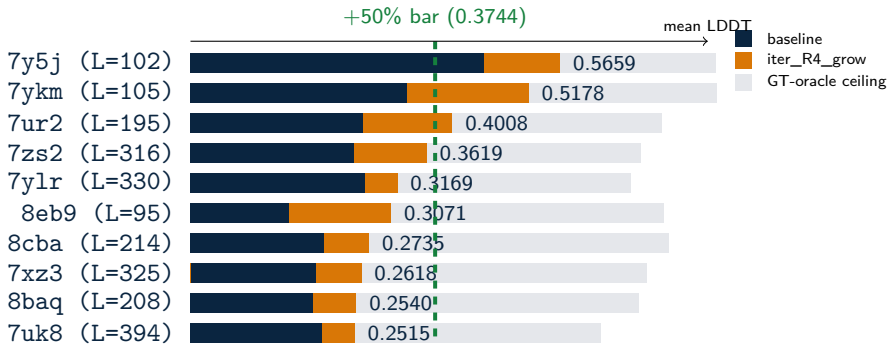
Contacts-only — distance-bin commits hurt LDDT (one-hot rows zero wrong-mode pairs). **Standard expected-distance readout** ($\sum p \cdot \text{midpoint}$) on the final distogram. **No post-hoc sharpening.**

Progression: knob $\rightarrow$ knob $\rightarrow$ knob



+10% bar (0.2746) cleared by sharpening alone. +15.94% by seeded + sharpen.
Iteration with growing K is the big lever past +25%. **+40.68% in 4373 s (3.16 $\times$ baseline, in budget).**

Per-protein results vs the +50% bar



4 / 10 proteins pass the +50% bar on this single algorithm. The hard misses (8baq, 7uk8, 7xz3, 8cba, 8eb9, 7ylr) all have huge oracle headroom (0.43–0.55) — model has the capacity, but its honest contact predictions remain sparse on these.

What didn't work

- ✗ **Sampled contact prefixes.** Model emits only 2–33 contacts before transitioning to `<distance>`; sampling-based seed selection has lower precision than top-K marginal. *+2.9% single, +0.3% M=5 average (averaging blurs distograms).*
- ✗ **Distance-bin commits.** Inject `<distance> ... <d_X.X>` statements for high-confidence pairs. Pair becomes one-hot row → LDDT zero when mode is wrong. *Net: 0.3503 vs 0.3511 (no change). Killed.*
- ✗ **$K = 2L$ seeded contacts.** Lower-precision tail seeds poison the readout. *-1.2% vs $K = L$. Precision > count.*
- ✗ **Mixture-of-distograms / per-pair max-confidence.** Variants aren't differently wrong — just less wrong, same direction. *Worse than the best single algorithm.*
- ✗ **Sharpening on top of iter_R4_grow.** $p' = \text{softmax}(\log p / T)$. $T=1.0$ (no sharpen) wins. *Sharpening only helps when distributions are high-entropy from bad context. Matches the GT-oracle pattern.*

Takeaways

- **Context > sampling.** Top- K from the marginal contact distribution beats sampling contact statements autoregressively at any T .
- **Iteration enriches the high-confidence contact pool.** Contacts above 0.5 prob grow 5–75 $\times$ after 3 iterations. Later rounds can afford larger K without losing precision.
- **Growing K matters.** Cautious early ($K = 0.5L$), bold late ($K = 2.5L$) beats fixed $K = L$ or $K = 2L$. Round 1's noisy bold picks would limit later rounds.
- **Precision > count.** $K = 2L$ always hurts. Distance commits hurt. What works is exactly the model's most-confident predictions, nothing extra.
- **Sharpening rescues bad context, not good context.** Big gain on naive (+9.7%); zero gain on oracle.
- **Contact prediction is the bottleneck, not the model.** GT-oracle reaches 0.72. For hard proteins, the model just doesn't honestly identify enough contacts at inference time.

Reproducer

```
# 1. Produce the baseline full-distogram prior
uv run python run_baseline.py --dtype bfloat16 --n-gpus 1
uv run python snapshot_distograms.py --to distogram_baseline_naive.npz

# 2. Run iter_R4_grow_05_10_15_25
uv run python run_iterative.py --dtype bfloat16 --n-gpus 1 \
    --algorithm iter_R4_grow_05_10_15_25 \
    --n-rounds 4 \
    --k-contacts-per-L-per-round 0.5 1.0 1.5 2.5 \
    --k-distances-per-L-per-round 0.0 0.0 0.0 0.0 \
    --min-contact-prob 0.1 \
    --prior-name distogram_baseline_naive.npz

# Result: mean LDDT 0.3511 (+40.68% over baseline 0.2496)
```

Branch: `exp/27-improved-inference-algorithm` · Narrative + rows: `RESULTS_LOG.md`,
`data/experiments.tsv`